Experiment No: 08

Aim: Implement a Deadlock management in Distributed systems

Theory:

Chandy-Misra-Hass deadlock detection algorithm

Chandy-Misra-Haas's distributed deadlock detection algorithm is an edge chasing algorithm to detect deadlock in distributed systems. It is also considered one of the best deadlock detection algorithms for distributed systems.

Algorithm:

Process of sending probe:

1. If process Pi is locally dependent on itself then declare a deadlock.

2. Else for all Pj and Pk check the following condition:

(a) Process Pi is locally dependent on process Pj

(b) Process Pj is waiting on process Pk

(c) Process Pj and process Pk are on different sites.

If all of the above conditions are true, send probe (i, j, k) to the home site of process Pk.

On the receipt of probe (i, j, k) at home site of process Pk: 1.

Process Pk checks the following conditions:

(a) Process Pk is blocked.

(b) dependentk[i] is false.

(c) Process Pk has not replied to all requests of process Pj

If all of the above conditions are found to be true then:

1. Set dependentk[i] to true.

2. Now, If k == i then, declare the Pi is deadlocked.

3. Else for all Pm and Pn check the following conditions:

(a) Process Pk is locally dependent on process Pm and

(b) Process Pm is waiting upon process Pn and

(c) Process Pm and process Pn are on different sites.

4. Send probe (i, m, n) to the home site of process Pn if the above conditions satisfy.

Thus, the probe message travels along the edges of the transaction wait-for (TWF) graph and when the probe message returns to its initiating process then it is said that deadlock has been detected.

Performance:

The algorithm requires at the most exchange of m(n-1)/2 messages to detect deadlock. Here, m is the number of processes and n is the number of sites.

The delay in detecting the deadlock is O(n).

Advantages:

1. There is no need for a special data structure. A probe message, which is very small and involves only 3 integers and a two-dimensional boolean array dependent is used in the deadlock detection process.

2. At each site, only a little computation is required and overhead is also low

3. Unlike other deadlock detection algorithm, there is no need to construct any graph or pass nor to pass graph information to other sites in this algorithm.

```python
from multiprocessing import Process, Value
from time import sleep

# Number of processes
n = 10

# Logical clocks for each process
clock = [Value('i', 0) for i in range(n)]

# Request flags for each process (1 if requesting, 0 otherwise)
request = [Value('i', 0) for i in range(n)]

def request_cs(pid):
    """ Function to request and enter the critical section """
    # Set request flag and increment logical clock
    request[pid].value = 1
    clock[pid].value += 1

    # Check other processes before entering critical section
    for j in range(n):
        if j != pid:
            while request[j].value == 1:  # If another process is requesting
                # Check timestamp priority (lower timestamp gets priority)
                if clock[j].value < clock[pid].value:
                    clock[pid].value += 1
                # If timestamps are the same, use process ID as tie-breaker
                elif (clock[j].value == clock[pid].value) and (j < pid):
                    clock[pid].value += 1
                else:
                    break  # Current process has priority, exit loop

    # Entering the critical section
    print(f"Process {pid} enters critical section at {clock[pid].value}")
    sleep(1)  # Simulate critical section work

    # Exiting the critical section
    print(f"Process {pid} exits critical section")
    request[pid].value = 0  # Reset request flag

if __name__ == "__main__":
    # Create process instances
    processes = [Process(target=request_cs, args=(i,)) for i in range(n)]

    # Start all processes
    for p in processes:
        p.start()

    # Wait for all processes to finish
    for p in processes:
        p.join()
```

```
PS C:\Users\LENOVO\OneDrive\Desktop\DC EXP> python app.py
Enter Initiator Site No. : 2

 S2 ==> S3    (2, 2, 3)
 S3 ==> S4    (2, 3, 4)
 S4 ==> S1    (2, 4, 1)
 S1 ==> S2    (2, 1, 2) -------- > DEADLOCK DETECTED
 S3 ==> S5    (2, 3, 5)
PS C:\Users\LENOVO\OneDrive\Desktop\DC EXP>
```

Conclusion: This experiment successfully implements the Chandy-Misra-Haas algorithm to detect deadlocks in a distributed system by identifying circular wait conditions in a wait-for graph.

```
from multiprocessing import Process, Value
from time import sleep

# Number of processes
n = 5  # Using 5 for a cleaner output, but works for 10

# Logical clocks for each process ('i' for integer, initialized to 0)
clock = [Value('i', 0) for i in range(n)]

# Request flags for each process (1 if requesting, 0 otherwise)
request = [Value('i', 0) for _ in range(n)]

def request_cs(pid):
    """Function to request and enter the critical section"""

    # 1. Set request flag and increment logical clock
    request[pid].value = 1
    clock[pid].value += 1

    print(f"Process {pid} is requesting CS with timestamp {clock[pid].value}")

    # 2. Check other processes before entering critical section
    for j in range(n):
        if j != pid:
            # If another process is requesting, we must check priority
            while request[j].value == 1:
                # Check timestamp priority (lower timestamp gets priority)
                if clock[j].value < clock[pid].value:
                    sleep(0.1) # Wait/Yield
                    continue
                # If timestamps are same, use process ID as tie-breaker
                elif (clock[j].value == clock[pid].value) and (j < pid):
                    sleep(0.1) # Wait/Yield
                    continue
                else:
                    # Current process has higher priority
                    break

    # 3. Entering the critical section
    print(f"--- Process {pid} ENTERS critical section at {clock[pid].value} ---")
    sleep(1) # Simulate work

    # 4. Exiting the critical section
    print(f"--- Process {pid} EXITS critical section ---")
    request[pid].value = 0 # Reset request flag

if __name__ == "__main__":
    # Create process instances
    processes = [Process(target=request_cs, args=(i,)) for i in range(n)]

    # Start all processes
    for p in processes:
        p.start()

    # Wait for all process
es to finish
    for p in processes:
        p.join()
```